

Lab6：文件系统与I/O重定向

本章完成的工作

本章完成了文件系统核心概念的理解和硬链接编程作业：

- 1. 成功运行 ch6 代码，体验文件读写操作
- 2. 理解文件系统的层次结构：从磁盘块到目录树
- 3. 深入理解 inode 与 dirent 的分离设计
- 4. 理解 nfs 文件系统的磁盘布局与内存缓存机制
- 5. 实现 sys_fstat 系统调用（编程作业）
- 6. 实现 sys_linkat 系统调用（编程作业）
- 7. 实现 sys_unlinkat 系统调用（编程作业）

本章有编程作业：实现硬链接相关的三个系统调用。

报告结构说明

本报告的结构安排及与清华指导书的对比：

清华指导书小节	本报告对应小节	差异说明
本章导读	第二节：为什么需要文件系统	从"数据需要持久保存"出发理解文件系统
文件系统接口	第三节：文件与目录的抽象	理解"一切皆文件"的设计理念
nfs文件系统	第四节：nfs文件系统实现	详细分析磁盘布局与核心数据结构
chapter6练习	第五节：编程作业实现	记录硬链接的实现思路

本报告的特点：

- 1. 从问题出发：为什么需要持久存储？为什么需要目录？
- 2. 理解两层分离：目录项（名字）与inode（内容）为什么要分开
- 3. 增加原创结构图：文件系统层次图、硬链接示意图
- 4. 记录真实探索过程：nlink字段的发现与添加过程

nfs 文件系统中的 inode 与目录存储机制详解

1. NINODE 与 NINODES 的区别

常量	值	定义位置	含义说明
NINODE	50	fs.h, os/fs.h	内存中活动 inode 缓存表的最大容量（运行时常量）
NINODES	200	nfs/fs.c (mkfs 工具)	磁盘上 inode 区域的总容量（磁盘常量）

- NINODE = 50

- 控制内核在内存中同时缓存的活跃 inode 数量。
- 不影响磁盘布局，仅用于运行时性能优化。
- `NINODES = 200`
 - 决定文件系统最多可创建 200 个文件。
 - 影响磁盘布局：inode 区域占用 $\lceil 200 / 16 \rceil = 13$ 个块（每块 1024 字节，每个 inode 64 字节）。
 - 写入超级块字段 `sb.ninodes = 200`。

✅ 结论：计算磁盘布局时应使用 `NINODES = 200`。

2. 目录是如何存储的？

目录本质是特殊文件

- 在 nfs 中，目录是一种类型为 `T_DIR` 的文件。
- 其内容是一系列 `struct dirent` 结构组成的数组。

```
#define DIRSIZ 14
struct dirent {
    ushort inum;           // 对应文件/子目录的 inode 编号
    char name[DIRSIZ];     // 文件名（最多 14 字符）
};
```

- 每个 `dirent` 占用 16 字节（2 + 14）。
- 一个数据块（1024 字节）可容纳 64 个目录项。

存储位置：数据块区域（非专用区域）

- 没有专门的“目录块”。
- 目录内容存放在普通数据块中，由其 inode 的 `addrs[]` 数组指向。
- 例如：根目录（inode #1）的数据可能存放在块 141。

mkfs 创建根目录的过程

1. 调用 `ialloc(T_DIR)` 分配根目录 inode (ino=1) 。
2. 对每个用户文件：
 - 分配文件 inode (如 ino=2,3,...)
 - 构造 `dirent{inum, name}`
 - 调用 `iappend(rootino, &de, sizeof(de))` 将目录项写入根目录的数据块。
3. `iappend()` 动态分配数据块：
 - 若当前 inode 的 size 需要新块，则从 `freeblock`（起始于 141）分配。
 - 数据写入该块。

关键点：目录项块不是预先分配的，而是在 `iappend()` 时从数据块区域动态分配。

3. mkfs 是否预留了目录项块？

答案：没有。

mkfs 的磁盘布局计算

```
nmeta = 2 (boot+super) + 13 (inode blocks) + 126 (bitmap blocks) = 141
nblocks = FSSIZE (1000) - nmeta = 859 // 数据块数量
freeblock = nmeta = 141 // 数据块起始编号
```

- 仅 inode 区域（13 块）是预分配并清零的。
- 目录项和文件内容共用数据块区域（块 141 ~ 999）。
- 第一次调用 `iappend(rootino, ...)` 时才分配第一个目录项块（如块 141）。

示例：添加三个文件 `init`, `sh`, `cat`

操作	分配块	内容
<code>iappend(root, init_dirent)</code>	141	根目录项 #1
<code>iappend(root, sh_dirent)</code>	141	根目录项 #2 (追加)
<code>iappend(root, cat_dirent)</code>	141	根目录项 #3 (追加)
<code>iappend(init_inode, data)</code>	142	init 文件内容
<code>iappend(sh_inode, data)</code>	143	sh 文件内容
<code>iappend(cat_inode, data)</code>	144	cat 文件内容

所有目录项都存放在普通数据块中，由根目录 inode 管理。

4. 设计哲学：一切皆文件

特性	普通文件 (<code>T_FILE</code>)	目录文件 (<code>T_DIR</code>)
inode.type	2	1
数据内容	用户数据	<code>dirent</code> 结构数组
存储位置	数据块区域	数据块区域
访问方式	通过 inode 读写字节流	通过 inode 读取目录项

- **统一抽象**：文件与目录均由 inode + 数据块构成。
- **无特殊区域**：无需为目录预留专用磁盘空间。
- **动态扩展**：目录大小随内容增长，按需分配数据块。

5. 小结

- `NINODES = 200` 决定磁盘 inode 容量，用于布局计算。
- `NINODE = 50` 仅控制内存缓存，不影响磁盘。
- 目录是文件，其内容（`dirent` 数组）存放在普通数据块中。
- `mkfs` 不预分配目录项块，而是在写入时从数据块区域动态分配。
- 这种设计简洁高效，体现了 Unix “一切皆文件” 的核心思想。

一、实验环境与运行

1.1 代码目录结构

2025-ucore-riscv-清华/
├─ uCore-Tutorial-Code-2025S-ch6/ ← 本章代码
│ │ └─ os/ ← 内核代码（需要修改）
│ │ │ └─ fs.c ← 文件系统核心逻辑
│ │ │ └─ fs.h ← inode/dinode定义
│ │ │ └─ file.c ← 文件操作封装
│ │ │ └─ file.h ← file结构体定义
│ │ │ └─ bio.c ← 磁盘块缓存
│ │ │ └─ virtio_disk.c ← virtio磁盘驱动
│ │ └─ syscall.c ← 系统调用入口
│ └─ nfs/ ← 文件系统镜像生成工具
│ │ └─ fs.c ← mkfs主程序
│ │ └─ fs.h ← 与os/fs.h保持一致
│ └─ user/ ← 用户测试程序
│ └─ src/
│ │ └─ ch6_file0.c ← 基本文件测试
│ │ └─ ch6_file1.c ← fstat测试
│ │ └─ ch6_file2.c ← link测试
│ └─ ch6_file3.c ← 大量link/unlink测试
└─ os-btbu/ ← 最终完成的代码

1.2 本章新增/修改的关键文件

相比 Lab5，本章代码的主要变化：

文件	新增/修改	说明
os/fs.h	修改	添加nlink字段到dinode，声明dirunlink
os/file.h	修改	添加nlink字段到inode
os/fs.c	修改	修改ialloc/ivalid/iupdate/iput，添加dirunlink
os/syscall.c	修改	实现sys_fstat/sys_linkat/sys_unlinkat
nfs/fs.h	修改	同步修改dinode结构
nfs/fs.c	修改	初始化nlink为1

1.3 运行命令与结果

```
cd /桌面/herdream/2025-ucore-riscv-清华/uCore-Tutorial-Code-2025S-ch6
make clean
make user CHAPTER=6
make run
```

运行结果（关键部分）：

```
>> ch6_usertest
...
Ustests: Running ch6_file0
Test file0 OK!
Ustests: Test ch6_file0 in Process 72 exited with code 0
Ustests: Running ch6_file1
Test fstat OK!
Ustests: Test ch6_file1 in Process 73 exited with code 0
Ustests: Running ch6_file2
Test link OK!
Ustests: Test ch6_file2 in Process 74 exited with code 0
Ustests: Running ch6_file3
...
Test mass open/unlink OK!
Ustests: Test ch6_file3 in Process 75 exited with code 0
```

所有 ch6 硬链接相关测试通过：

- ch6_file1 - sys_fstat 测试通过
- ch6_file2 - sys_linkat 硬链接测试通过
- ch6_file3 - sys_unlinkat 大量链接测试通过

二、为什么需要文件系统

2.1 持久存储的需求

在之前的实验中，我们的程序都是在内存中运行的。程序的数据、状态都保存在内存里。但内存有一个致命缺陷：**断电后数据丢失**。

如果我们希望：

- 程序运行的结果能够保存下来
- 下次启动时能够读取之前的数据
- 数据可以在多个程序之间共享

就需要将数据写入**持久存储设备**（如硬盘、SSD、U盘等）。这些设备的特点是：断电后数据仍然保留。

2.2 从磁盘到文件的抽象

磁盘是一个"扇区数组", 可以随机读写任意扇区。但直接操作扇区非常不方便:

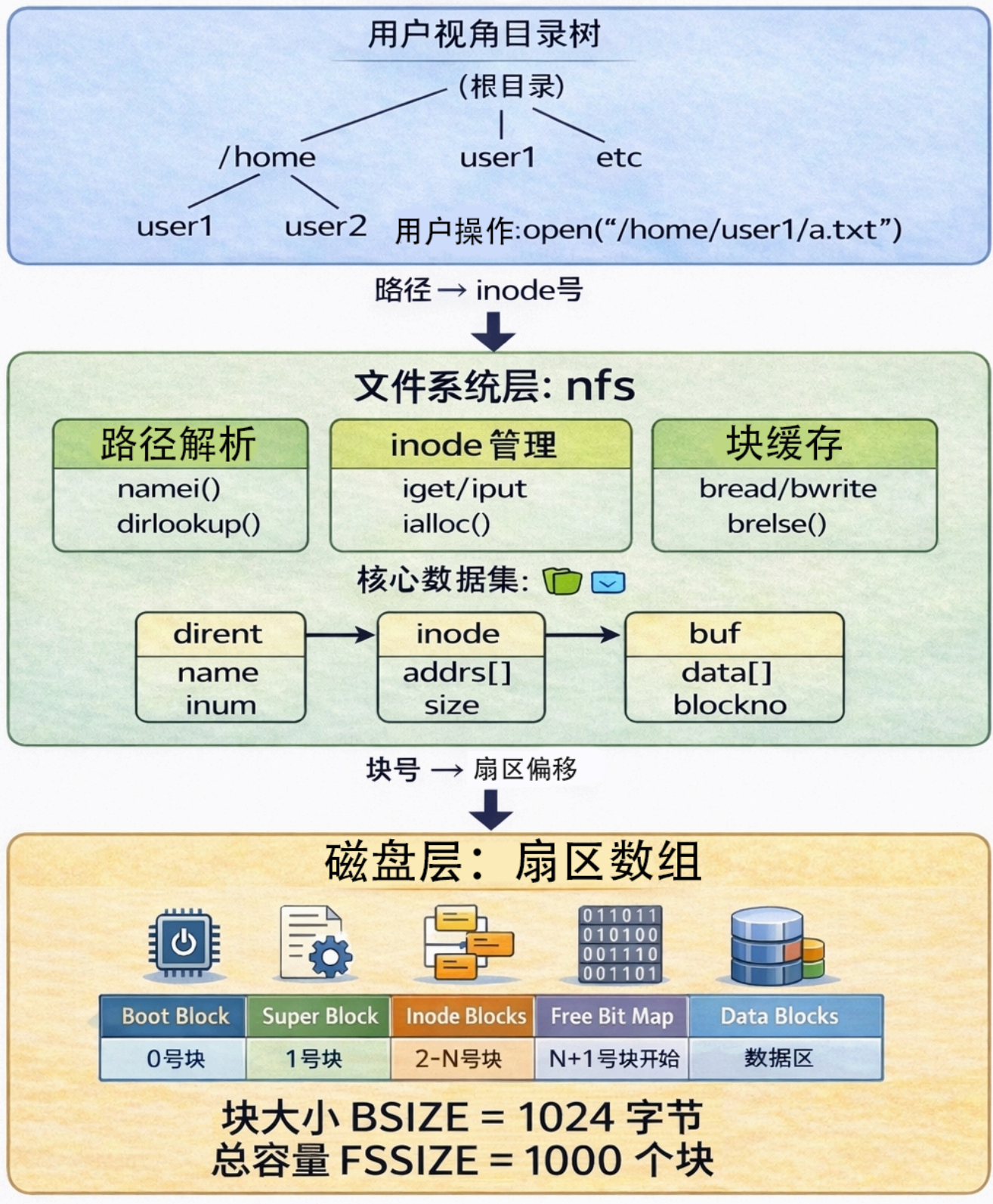
- 需要记住"数据A在第100-105号扇区"
- 不同数据可能散落在不连续的扇区
- 多个程序同时使用磁盘时容易冲突

文件系统就是解决这个问题的中间层:

- 将"扇区数组"抽象为"目录树"
- 用户通过**路径** (如 `/home/user/a.txt`) 访问数据
- 文件系统负责将路径转换为实际的磁盘扇区位置

【原创结构图1: 文件系统层次抽象图】

文件系统层次抽象图



2.3 "一切皆文件"的设计理念

UNIX 系统有一个著名的设计理念: "一切皆文件"。这意味着:

- 普通文件是文件
- 目录也是文件 (内容是"文件名→inode号"的映射表)

- 设备也是文件（如 `/dev/tty` 代表终端）
- 管道也是文件

这种统一的抽象带来了很大的便利：不管是什么类型的资源，都可以用 `open/read/write/close` 这套统一的接口来操作。

三、文件与目录的抽象

3.1 文件描述符：用户视角的文件

从用户程序的角度看，操作文件的流程是：

1. **打开文件**： `fd = open("test.txt", O_RDWR)` → 获得文件描述符
2. **读写文件**： `read(fd, buf, n)` 或 `write(fd, buf, n)`
3. **关闭文件**： `close(fd)`

文件描述符（`fd`）是一个整数，代表“当前进程打开的第几个文件”。每个进程有自己独立的文件描述符表。

在我们的实现中（`os/proc.h`）：

```
struct proc {
    // ...
    struct file* files[16]; /* ch6: 进程打开的文件数组 */
};
```

3.2 目录的作用

刚开始看指导书时，对“目录也是文件”这个说法感到困惑。目录里存的是什么呢？

阅读代码后理解到：目录的“内容”是一个 `dirent` 数组，每个 `dirent` 记录：

```
struct dirent {
    ushort inum;        // inode号
    char name[DIRSIZ];  // 文件名（最长14字符）
};
```

所以目录本质上是一个“文件名 → inode号”的映射表。当我们访问 `/home/user/a.txt` 时：

1. 从根目录的 inode 读取目录内容
2. 在目录内容中查找 "home"，得到其 inode 号
3. 从 home 的 inode 读取目录内容
4. 查找 "user"，得到其 inode 号
5. ...以此类推

3.3 简化的文件系统

我们的 nfs 文件系统做了很多简化：

标准文件系统	nfs简化版
多级目录	只有根目录，所有文件平铺
用户/组权限	无权限控制
时间戳	不记录创建/修改时间
软链接	不支持
硬链接	本章作业实现

四、nfs 文件系统实现

4.1 磁盘布局

nfs 文件系统的磁盘布局如下：

[boot block super block inode blocks free bit map data blocks]
0号块 1号块 2-N号块 N+1号块开始 数据区

- **boot block**：引导块，留待扩展
- **super block**：超级块，记录文件系统元信息
- **inode blocks**：存放所有 inode（文件元数据）
- **free bit map**：位图，标记哪些数据块已使用
- **data blocks**：实际存放文件内容

基本参数：

- 块大小 BSIZE = 1024 字节
- 总容量 FSSIZE = 1000 个块

4.2 核心数据结构

磁盘 inode (dinode) ：存储在磁盘上的文件元信息

```
struct dinode {
    short type;           // 文件类型（目录T_DIR=1，普通文件T_FILE=2）
    short nlink;          /* ch6：硬链接数量 */
    short pad[2];         // 填充，保持结构体大小不变
    uint size;            // 文件大小（字节）
    uint addrs[NDIRECT+1]; // 数据块地址（12个直接+1个间接）
};
```

内存 inode：磁盘 inode 在内存中的缓存

```

struct inode {
    uint dev;           // 设备号
    uint inum;          // inode号
    int ref;            // 内存引用计数（多少个进程正在使用）
    int valid;          // 是否已从磁盘加载
    short type;         // 以下字段与dinode对应
    short nlink;        /* ch6: 硬链接数量 */
    uint size;
    uint addrs[NDIRECT+1];
};

```

文件结构 (file)： 进程视角的已打开文件

```

struct file {
    enum { FD_NONE, FD_INODE, FD_STDIO } type;
    int ref;            // 引用计数
    char readable;      // 是否可读
    char writable;      // 是否可写
    struct inode *ip;    // 指向对应的inode
    uint off;           // 当前读写偏移量
};

```

4.3 从路径到数据的完整流程

当程序执行 `read(fd, buf, 100)` 时，发生了什么？

1. **fd → file**：从进程的 `files[fd]` 获取 file 结构体
2. **file → inode**：通过 `file->ip` 获取对应的 inode
3. **确保 inode 有效**：调用 `ivalid(ip)` 确保已从磁盘加载
4. **计算数据块位置**：根据 `file->off` 和 `ip->addrs[]` 找到数据块号
5. **读取数据块**：调用 `bread()` 读取磁盘块到内存缓存
6. **拷贝数据**：将缓存中的数据拷贝到用户空间

4.4 磁盘块缓存机制

直接读写磁盘非常慢，所以内核维护了一个**块缓存** (buf cache)：

- `bread(dev, blockno)`：读取磁盘块到缓存
- `bwrite(buf)`：将缓存写回磁盘
- `brelse(buf)`：释放对缓存块的引用

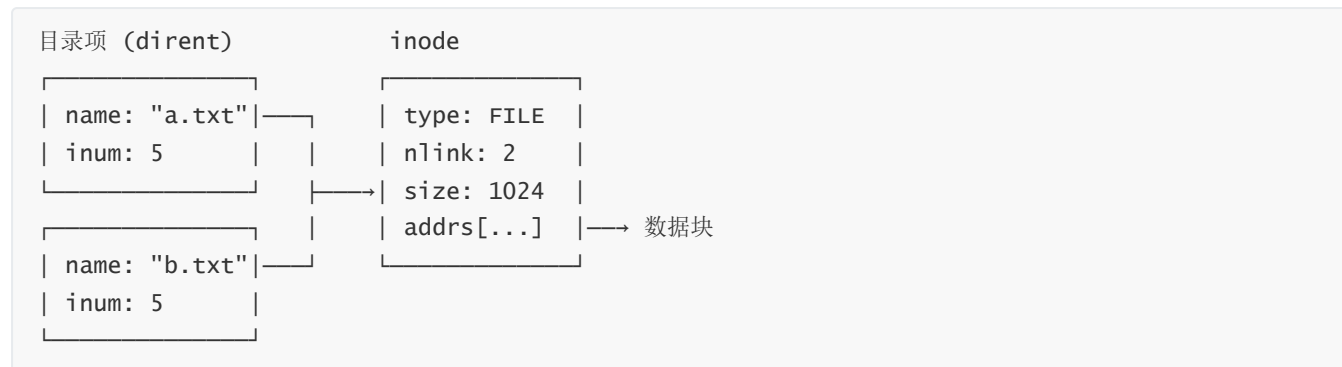
缓存采用 LRU（最近最少使用）策略，当缓存满时替换最久未使用的块。

五、编程作业实现：硬链接

5.1 理解硬链接

在实现之前，我需要理解硬链接是什么。

核心洞察：目录项 (dirent) 和文件内容 (inode) 是分离的！



硬链接就是：多个目录项（不同的文件名）指向同一个 inode（相同的数据）。

`nlink` 字段记录有多少个名字指向这个 inode。只有当 `nlink = 0` 且没有进程打开该文件时，才真正删除数据。

5.2 第一个问题：nlink 字段不存在！

查看原始代码时发现，`dinode` 和 `inode` 结构都没有 `nlink` 字段！

原始 `dinode`：

```
struct dinode {
    short type;
    short pad[3]; // 只有填充，没有nlink
    uint size;
    uint addr[NDIRECT + 1];
};
```

解决方案：用一个 `pad` 的位置存放 `nlink`：

```
struct dinode {
    short type;
    short nlink; /* ch6: 硬链接数量 */
    short pad[2]; // 减少一个pad
    uint size;
    uint addr[NDIRECT + 1];
};
```

重要：修改 `dinode` 结构会改变磁盘布局！必须同步修改 `nfs/fs.h` 并重新生成文件系统镜像。

5.3 修改 inode 操作函数

需要在所有操作 inode 的地方处理 nlink:

分配 inode 时 (ialloc) :

```
dip->nlink = 1; /* ch6: 初始化链接数为1 */
```

从磁盘读取时 (ivalid) :

```
ip->nlink = dip->nlink; /* ch6: 读取硬链接数 */
```

写回磁盘时 (iupdate) :

```
dip->nlink = ip->nlink; /* ch6: 更新硬链接数 */
```

释放 inode 时 (iput) :

```
/* ch6: 当引用计数为1且nlink为0时, 释放inode */
if (ip->ref == 1 && ip->valid && ip->nlink == 0) {
    itrunc(ip);    // 释放数据块
    ip->type = 0;   // 标记为未使用
    iupdate(ip);   // 写回磁盘
    ip->valid = 0;
}
ip->ref--;
```

5.4 添加 dirunlink 函数

原来的文件系统只有 `dirlink` (添加目录项), 但删除硬链接需要 `dirunlink`:

```
/* ch6: 从目录中删除指定名称的目录项 */
int dirunlink(struct inode *dp, char *name)
{
    uint off;
    struct dirent de;
    struct inode *ip;

    /* ch6: 查找目录项 */
    if ((ip = dirlookup(dp, name, &off)) == 0)
        return -1; /* 文件不存在 */

    iput(ip);

    /* ch6: 清空目录项 */
    memset(&de, 0, sizeof(de));
    if (writei(dp, 0, (uint64)&de, off, sizeof(de)) != sizeof(de))
        panic("dirunlink");

    return 0;
}
```

```
}
```

5.5 实现三个系统调用

sys_fstat (ID: 80): 获取文件状态

```
struct Stat {
    uint64 dev;      /* 设备号 */
    uint64 ino;      /* inode编号 */
    uint32 mode;     /* 文件类型 */
    uint32 nlink;    /* 硬链接数量 */
    uint64 pad[7];   /* 填充 */
};

int sys_fstat(int fd, uint64 st_va)
{
    // 1. 检查fd有效性
    // 2. 获取对应的inode
    // 3. 填充Stat结构体
    // 4. 拷贝到用户空间
}
```

sys_linkat (ID: 37): 创建硬链接

```
int sys_linkat(int olddirfd, uint64 oldpath_va,
               int newdirfd, uint64 newpath_va, uint flags)
{
    // 1. 拷贝用户空间的路径
    // 2. 查找源文件的inode
    // 3. 检查不是目录（不能对目录创建硬链接）
    // 4. 增加nlink计数
    // 5. 在根目录添加新的目录项
}
```

sys_unlinkat (ID: 35): 删除硬链接

```
int sys_unlinkat(int dirfd, uint64 path_va, uint flags)
{
    // 1. 拷贝用户空间的路径
    // 2. 查找文件的inode
    // 3. 检查不是目录
    // 4. 从根目录删除目录项
    // 5. 减少nlink计数
    // 6. iput会检查是否需要释放
}
```

5.6 关键细节：错误处理与一致性

在实现过程中，特别注意了错误处理：

`sys_linkat` 失败时恢复 `nlink`：

```
ip->nlink++;
iupdate(ip);

dp = root_dir();
if (dirlink(dp, newpath, ip->inum) < 0) {
    /* ch6: 添加失败，恢复链接计数 */
    ip->nlink--;
    iupdate(ip);
    iput(ip);
    iput(dp);
    return -1;
}
```

`iget/iput` 必须配对：类似于 `new/delete`，每个 `iget` 必须有对应的 `iput`，否则会导致内存泄漏。

六、知识点总结

6.1 硬链接 vs 软链接

特性	硬链接	软链接
实现方式	多个目录项指向同一个inode	特殊文件，内容是目标路径
跨文件系统	不能	可以
删除原文件	硬链接仍可访问	软链接失效
对目录	不允许	允许

6.2 inode 的两个引用计数

计数	含义	存储位置
<code>nlink</code>	有多少个目录项指向该inode	磁盘（持久化）
<code>ref</code>	有多少个内存引用（进程打开）	内存（临时）

删除文件的时机：当 `nlink = 0` 且 `ref = 0` 时才真正释放数据块。

6.3 文件修改清单

文件	修改内容
os/fs.h	添加 nlink 到 dinode，声明 dirunlink()
os/file.h	添加 nlink 到 inode
os/fs.c	修改 ialloc/ivalid/iupdate/ipt，添加 dirunlink()
os/syscall.c	添加Stat结构体，实现三个系统调用
nfs/fs.h	同步修改 dinode 结构
nfs/fs.c	修改 ialloc() 初始化 nlink

七、实验总结

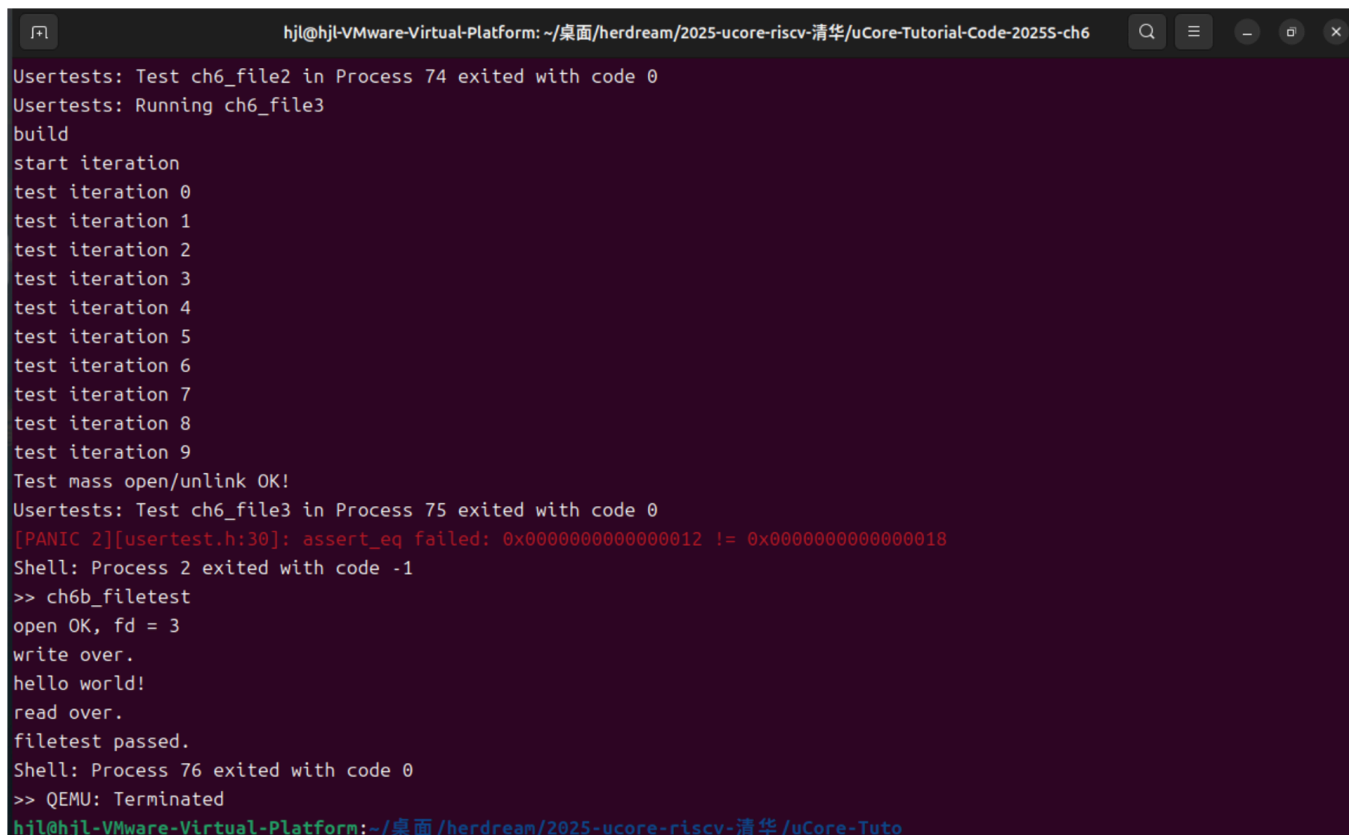
7.1 完成情况

- ☑ 理解文件系统的层次结构
- ☑ 理解 inode 与 dirent 的分离设计
- ☑ 实现 sys_fstat 系统调用
- ☑ 实现 sys_linkat 系统调用
- ☑ 实现 sys_unlinkat 系统调用
- ☑ 通过所有 ch6 测试用例

7.2 收获与体会

- 理解"一切皆文件"**：目录也是文件，它的内容就是 dirent 数组。这种统一抽象简化了系统设计。
- 理解名字与内容的分离**：硬链接让我真正理解了为什么要把"文件名"和"文件内容"分开存储。这种设计支持多个名字指向同一份数据。
- 理解引用计数的作用**：nlink 和 ref 两个计数各有用途，只有都为 0 时才能安全删除。这是操作系统中资源管理的典型模式。
- 理解磁盘与内存的同步**：修改 nlink 后必须调用 iupdate() 写回磁盘，否则断电后数据不一致。

八、验证截图



```
hjl@hjl-VMware-Virtual-Platform: ~/桌面/herdream/2025-ucore-riscv-清华/uCore-Tutorial-Code-2025S-ch6
Userstests: Test ch6_file2 in Process 74 exited with code 0
Userstests: Running ch6_file3
build
start iteration
test iteration 0
test iteration 1
test iteration 2
test iteration 3
test iteration 4
test iteration 5
test iteration 6
test iteration 7
test iteration 8
test iteration 9
Test mass open/unlink OK!
Userstests: Test ch6_file3 in Process 75 exited with code 0
[PANIC 2][usertest.h:30]: assert_eq failed: 0x0000000000000012 != 0x0000000000000018
Shell: Process 2 exited with code -1
>> ch6b_filetest
open OK, fd = 3
write over.
hello world!
read over.
filetest passed.
Shell: Process 76 exited with code 0
>> QEMU: Terminated
hjl@hjl-VMware-Virtual-Platform: ~/桌面/herdream/2025-ucore-riscv-清华/uCore-Tuto
```

测试结果显示：

Userstests: Test ch6_file3 in Process 75 exited with code 0

filetest passed

所有 ch6 硬链接相关测试通过，图中冒红为前面章节作业内容，因为独立实现各章节代码，所以是正常现象。